

Interpolation Methods

Newton, Lagrange, Hermite and the Runge phenomenon

Made by **Otmane BELKHAIR**

1 Comparing the three methods

We test the methods on a simple function and its derivative:

$$f(x) = x^2 + 2x + 1, \quad f'(x) = 2x + 2.$$

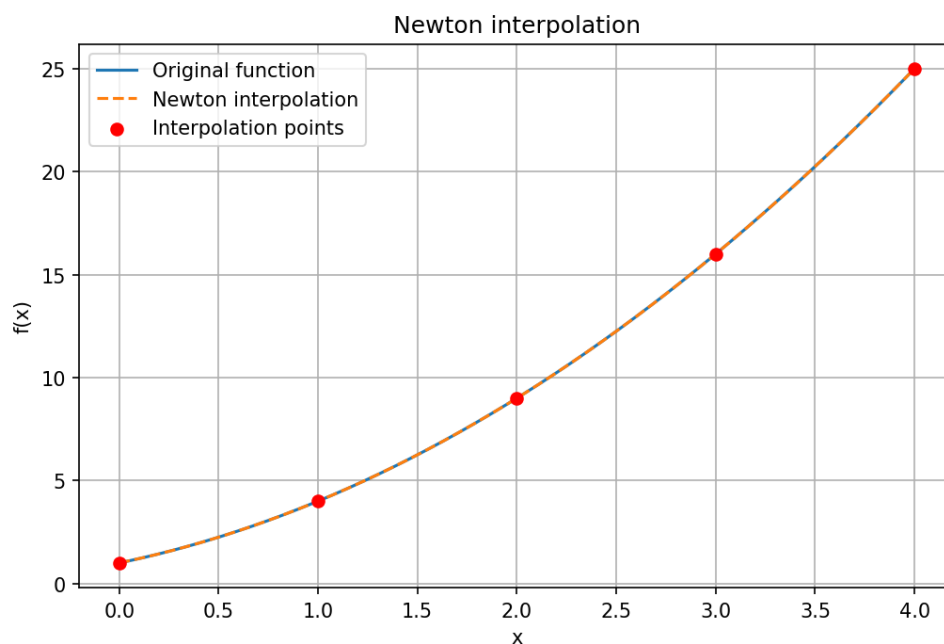
Newton and Lagrange use only the values of f . Hermite also uses the values of f' . The interpolation functions are given in the Appendix. The listings below show the core of each script; the full scripts also draw and save the figure.

1.1 Newton

We use the points $x = [0, 1, 2, 3, 4]$ with $f(x) = [1, 4, 9, 16, 25]$.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from newton import newton_interpolation
5
6 # Original function
7 def f(x):
8     return x**2 + 2*x + 1
9
10 # Interpolation points and their images
11 x = [0, 1, 2, 3, 4]
12 y = [f(xi) for xi in x]
13
14 # Dense grid to obtain a smooth curve
15 x_plot = np.linspace(0, 4, 300)
16 y_original = [f(xi) for xi in x_plot]
17 y_newton = [newton_interpolation(x, y, xi)[0] for xi in x_plot]
```

Code 1: Newton interpolation (core of the script)



Newton interpolation with 5 points. The two curves are identical.

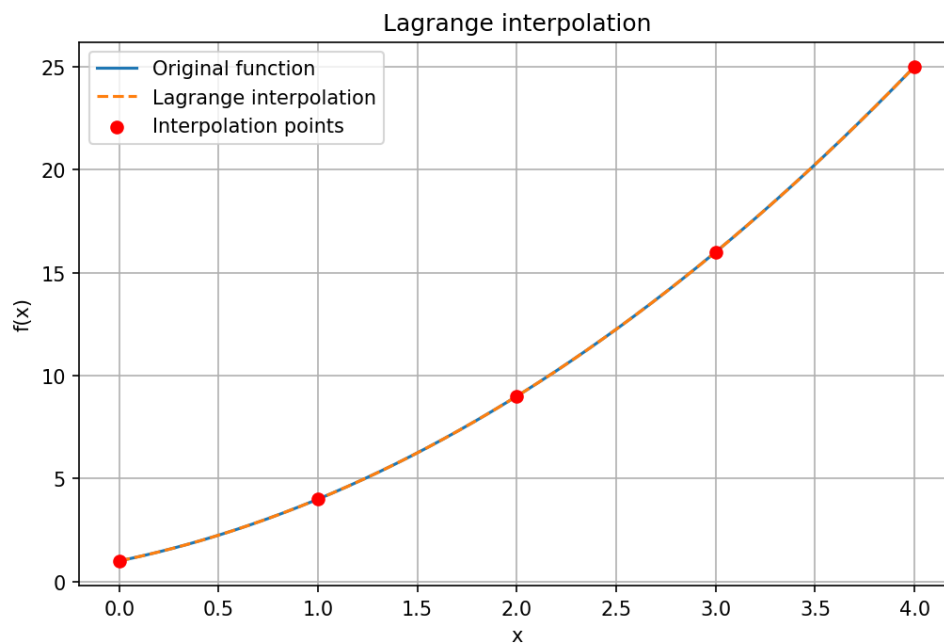
Five points give a polynomial of degree at most 4. Our function has degree 2 and passes through all five points. Since this polynomial is unique, it must be f itself.

1.2 Lagrange

Same function, same points.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from lagrange import lagrange_interpolation
5
6 # Original function
7 def f(x):
8     return x**2 + 2*x + 1
9
10 # Interpolation points and their images
11 x = [0, 1, 2, 3, 4]
12 y = [f(xi) for xi in x]
13
14 # Dense grid to obtain a smooth curve
15 x_plot = np.linspace(0, 4, 300)
16 y_original = [f(xi) for xi in x_plot]
17 y_lagrange = [lagrange_interpolation(x, y, xi) for xi in x_plot]
```

Code 2: Lagrange interpolation (core of the script)



Lagrange interpolation with 5 points. Again, the curves match.

The result is the same as Newton's, for the same reason.

1.3 Hermite

Here we use three points $x = [0, 2, 4]$, their values $f(x) = [1, 9, 25]$ and their slopes $f'(x) = [2, 6, 10]$.

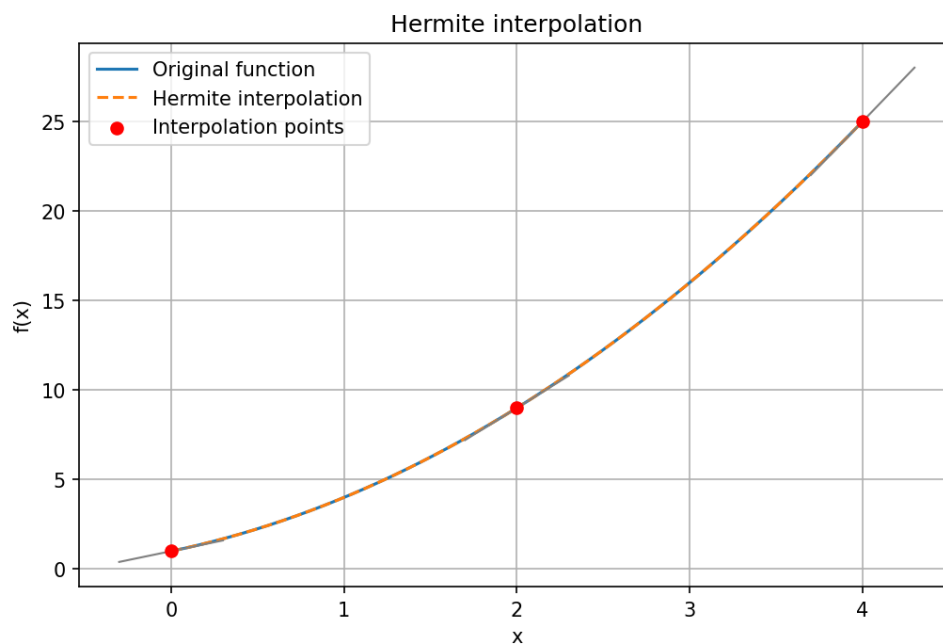
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from hermite import hermite_interpolation
5
6 # Original function and its derivative
7 def f(x):
8     return x**2 + 2*x + 1
```

```

9
10 def df(x):
11     return 2*x + 2
12
13 # Interpolation points, values and derivative values
14 x = [0, 2, 4]
15 y = [f(xi) for xi in x]
16 dy = [df(xi) for xi in x]
17
18 # Dense grid to obtain a smooth curve
19 x_plot = np.linspace(0, 4, 300)
20 y_original = [f(xi) for xi in x_plot]
21 y_hermite = [hermite_interpolation(x, y, dy, xi) for xi in x_plot]

```

Code 3: Hermite interpolation (core of the script)



Hermite interpolation. The grey lines show the given slopes.

Three points with slopes give $2 \times 3 = 6$ conditions, so the polynomial has degree at most 5. The function f satisfies all of them, so Hermite also returns f exactly.

Newton vs Lagrange vs Hermite. Newton and Lagrange give *the same polynomial*; they are just written in different ways. Newton is easier when you add a new point. Hermite gives a different polynomial in general, because it also matches the slopes.

2 The Runge phenomenon

The **Runge phenomenon** is the appearance of large oscillations near the ends of an interval when a high-degree polynomial interpolates a function at equally spaced points. More points does not always mean a better fit. Take

$$f(x) = \frac{1}{1 + 25x^2} \quad \text{on } [-1, 1],$$

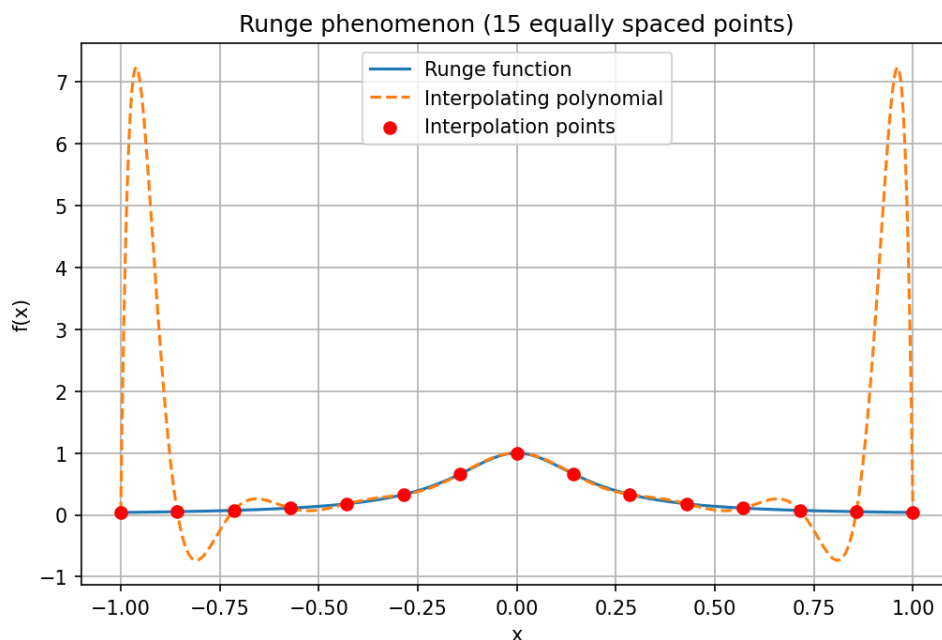
and interpolate it with 15 equally spaced points.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from lagrange import lagrange_interpolation
5
6 # Runge function
7 def f(x):
8     return 1 / (1 + 25*x**2)
9
10 # Number of interpolation points and equally spaced nodes
11 n = 15
12 x = np.linspace(-1, 1, n)
13 y = [f(xi) for xi in x]
14
15 # Dense grid
16 x_plot = np.linspace(-1, 1, 500)
17 y_original = [f(xi) for xi in x_plot]
18 y_lagrange = [lagrange_interpolation(x, y, xi) for xi in x_plot]

```

Code 4: Runge phenomenon (core of the script)



15 equally spaced points. The polynomial oscillates near the ends (maximum error about 7.2).

The polynomial fits well in the middle but swings wildly near -1 and 1 . This happens because the high derivatives of this function grow very fast.

Runge phenomenon: description. The Runge phenomenon happens when a high-degree polynomial is fitted through many equally spaced points. The polynomial passes through every point, but it oscillates strongly near the ends of the interval, while staying close to the function in the middle. In our example, with 15 points the largest error was about 7.2, much bigger than the function itself, which never goes above 1. It shows that more points does not always give a better approximation.